

Rapport de projet

Titre Professionnel Concepteur Développeur d'Applications

Lifequest - Application de gestion financière personnelle

Candidat : Antonin MINGAM

Titre visé : Concepteur Développeur d'Applications Niveau 6

Référentiel : TP-01281 millésime 04 (24/05/2023)

Sommaire

1. [Présentation du candidat et du contexte de formation](#)
2. [Présentation du projet Lifequest](#)
3. 2.1 [Contexte et objectifs](#)
4. 2.2 [Stack technique](#)
5. 2.3 [Architecture générale](#)

BLOC 1 — Développer une application sécurisée

1. [CP1 — Installer et configurer son environnement de travail en fonction du projet](#)
2. [CP2 — Développer des interfaces utilisateur](#)
 3. 4.1 [Conception des interfaces avec Phoenix LiveView](#)
 4. 4.2 [Charte graphique et responsive design \(Tailwind CSS / daisyUI\)](#)
 5. 4.3 [Accessibilité \(RGAA\)](#)
 6. 4.4 [Conformité RGPD — Mentions légales](#)
 7. 4.5 [Tests des composants d'interface](#)
 8. 4.6 [Sécurité des interfaces \(XSS, CSRF\)](#)
9. [CP3 — Développer des composants métier](#)
 10. 5.1 [Architecture en contextes \(Finances, Accounts\)](#)
 11. 5.2 [Développement défensif et gestion des erreurs](#)
 12. 5.3 [Communication en temps réel avec Phoenix PubSub](#)
 13. 5.4 [Tests unitaires des composants métier](#)
 14. 5.5 [Sécurité des composants métier](#)
15. [CP4 — Contribuer à la gestion d'un projet informatique](#)
 16. 6.1 [Méthode de développement itérative](#)
 17. 6.2 [Planification et suivi des tâches \(Jira\)](#)
 18. 6.3 [Outils collaboratifs \(GitHub, conventions de commits\)](#)
 19. 6.4 [Qualité du code \(Credo, mix format, CI\)](#)

BLOC 2 — Concevoir et développer une application sécurisée organisée en couches

1. [CP5 — Analyser les besoins et maquetter une application](#)
2. 7.1 [Analyse du cahier des charges et identification des besoins](#)
3. 7.2 [User stories](#)
4. 7.3 [Maquettes et enchaînement des écrans](#)
5. [CP6 — Définir l'architecture logicielle d'une application](#)
6. 8.1 [Architecture multicouche \(web / contextes / schémas\)](#)
7. 8.2 [Choix du framework et de l'ORM \(Phoenix / Ecto\)](#)
8. 8.3 [Stratégie de sécurité par couche](#)
9. 8.4 [Éco-conception](#)
10. [CP7 — Concevoir et mettre en place une base de données relationnelle](#)
11. 9.1 [Modèle Conceptuel de Données \(MCD\)](#)
12. 9.2 [Modèle Logique de Données \(MLD\)](#)
13. 9.3 [Migrations Ecto et scripts de création](#)
14. 9.4 [Sécurité et droits d'accès](#)
15. 9.5 [Jeu d'essai](#)
16. [CP8 — Développer des composants d'accès aux données SQL et NoSQL](#)
 - 10.1 [Requêtes Ecto \(CRUD sécurisé\)](#)
 - 10.2 [Gestion de l'intégrité et des conflits d'accès \(transactions\)](#)
 - 10.3 [Validation et contrôle des entrées côté serveur](#)
 - 10.4 [Tests unitaires et de sécurité des accès aux données](#)
 - 10.5 [Protection contre l'injection SQL](#)

BLOC 3 — Préparer le déploiement d'une application sécurisée

1. [CP9 — Préparer et exécuter les plans de tests d'une application](#)
 - 11.1 [Plan de tests](#)
 - 11.2 [Tests unitaires \(ExUnit\)](#)

- 11.3 [Tests d'intégration \(LiveView\)](#)
 - 11.4 [Tests de sécurité](#)
 - 11.5 [Résultats et compte-rendu de tests](#)
2. [CP10 — Préparer et documenter le déploiement d'une application](#)
- 12.1 [Environnements \(développement, test, production\)](#)
 - 12.2 [Procédure de déploiement](#)
 - 12.3 [Scripts de déploiement et migrations](#)
 - 12.4 [Variables d'environnement et configuration](#)
3. [CP11 — Contribuer à la mise en production dans une démarche DevOps](#)
- 13.1 [Pipeline CI/CD \(GitHub Actions\)](#)
 - 13.2 [Conteneurisation \(Docker\)](#)
 - 13.3 [Outils de qualité de code \(Credo, mix format\)](#)
 - 13.4 [Automatisation des tests](#)
 - 13.5 [Interprétation des rapports CI](#)

Compétences transversales

1. [Communiquer en français et en anglais](#)
- 14.1 [Documentation technique en anglais](#)
 - 14.2 [Lecture de documentation technique en anglais \(niveau B1\)](#)
2. [Mettre en œuvre une démarche de résolution de problème](#)
- 15.1 [Refactoring du modèle de données \(LQ-5\)](#)
 - 15.2 [Encodage des apostrophes dans les tests LiveView](#)
 - 15.3 [Diagnostic d'une CI rouge inattendue](#)
3. [Apprendre en continu — Veille technologique](#)
- 16.1 [Sources suivies](#)
 - 16.2 [Format et méthode](#)
 - 16.3 [Exemple concret : adoption de Tailwind CSS v4](#)

4. [Conclusion et bilan de compétences](#)

- 17.1 [Bilan technique](#)
- 17.2 [Bilan personnel](#)
- 17.3 [Suite et perspectives](#)

5. [Annexes](#)

- A. Diagramme MCD / MPD
- B. Maquettes des interfaces
- C. Plan de tests complet
- D. Extraits de code commentés
- E. Captures d'écran de l'application

1. Présentation du candidat et du contexte de formation

Avant d'entrer en formation, mon parcours étudiant et professionnel a été un long processus exploratoire. Après une scolarité exemplaire, j'ai traversé plusieurs filières classiques qui ne me correspondaient pas, avant de faire le choix d'une carrière mêlant technique et artistique, en tant qu'ingénieur du son. J'ai exercé cette activité pendant 5 ans en autoentreprise, acquérant des compétences techniques et transversales en autonomie et en gestion d'entreprise.

Mon choix de changer de direction arrive à la rencontre de plusieurs événements simultanés dans ma vie d'entrepreneur :

- la sensation d'avoir atteint un plafond de verre
- la frustration de ne pas avoir de diplôme qualifiant
- l'arrivée imminente de mes 30 ans
- la rencontre avec le monde du développement

En effet, malgré une rapide découverte au cours de mon adolescence, je n'avais jamais vraiment compris ce monde si vaste et riche. Lors de mon parcours entrepreneurial, j'ai été amené à rédiger des newsletters en HTML/CSS d'abord, puis à vouloir refondre mon site web professionnel, initialement créé avec un outil no-code. J'ai fini par consacrer une grande partie de mon temps libre à l'apprentissage des bases du développement, grâce aux cours en ligne d'OpenClassrooms puis de l'université d'Harvard. Cette montée en compétences

progressive m'a convaincu de mon attrait pour le développement, et de ma motivation à concrétiser ces nouvelles compétences grâce à des études traditionnelles.

C'est dans la suite logique de cette formation autodidacte que j'ai décidé d'intégrer la 2iAcademy en Conception et Développement d'Applications, en alternance chez Frixel, afin de renforcer mes compétences et d'en acquérir de nouvelles en conception, ainsi que d'obtenir un diplôme reconnu et qualifiant.

À l'issue de cette formation, je poursuivrai au sein de l'ESGI Paris en Mastère Big Data & Intelligence Artificielle dans le but de me spécialiser dans un secteur en forte croissance. À plus long terme, je me réserve le droit de retrouver le chemin de l'entrepreneuriat, fort d'une double compétence technique et d'un diplôme reconnu.

2. Présentation du projet Lifequest

2.1 Contexte et objectifs

Durant mon activité entrepreneuriale, il était très compliqué de me projeter dans l'avenir à moyen ou long terme. Mon arrivée dans un univers professionnel stable m'a permis de découvrir la gestion d'épargne à long terme, les objectifs financiers et les placements financiers. En explorant le sujet, j'ai découvert beaucoup d'applications de gestion financière et de placements financiers, mais aucune qui permettait de faire les deux à la fois, en plus d'offrir du conseil en gestion de patrimoine. Des applications comme Bankin' ou Linxo se concentrent sur le suivi bancaire et la catégorisation des dépenses, tandis que Finary ou Trade Republic se positionnent sur la gestion de placements. Aucune n'articule les deux en les ancrant autour d'objectifs de vie personnels, avec un moteur de simulation intégré. Un ami travaillant dans la finance m'avait fait part de son idée d'application de gestion et de conseil financiers, et une connaissance japonaise m'a fait découvrir des outils de gestion financière répandus au Japon. Ces différents échanges ont fait émerger l'idée de Lifequest, avec une approche innovante articulée autour des objectifs de vie de l'utilisateur, comme un achat de maison ou la volonté d'avoir un enfant. L'utilisateur renseigne sa situation financière actuelle, une estimation de ses revenus et de ses dépenses mensuels. Cela nous permet de prendre une photo de son patrimoine et d'effectuer des projections du futur simplement. Il définit

ensuite un objectif représenté par un montant cible et une échéance. À partir de ces données, l'application génère des recommandations personnalisées : réduction des dépenses non essentielles, placements ponctuels ou réguliers, avec un niveau de risque adapté à la complexité de l'objectif et à la situation financière déclarée.

L'application cible des particuliers souhaitant gérer leur budget personnel et planifier des objectifs financiers à moyen ou long terme, sans nécessiter de connaissances préalables en finance.

2.2 Stack technique

- Backend : Elixir 1.18 / OTP 27, Phoenix 1.8, Phoenix LiveView 1.1, Ecto
- Base de données : PostgreSQL 16
- Frontend : Tailwind CSS v4, daisyUI, templates HEx (pas de framework JS séparé)
- Tests : ExUnit, ConnCase, Phoenix.LiveViewTest
- CI : GitHub Actions

Elixir est un langage fonctionnel, concurrent et tolérant aux pannes, qui s'exécute sur la BEAM, la machine virtuelle d'Erlang éprouvée depuis des décennies dans les télécoms. Phoenix est le framework web Elixir de référence, qui embarque LiveView pour la construction d'interfaces réactives côté serveur.

Le choix de cette stack technique s'est présenté comme une évidence, car c'est celle que j'ai apprise et sur laquelle j'ai travaillé durant la majorité de mon alternance chez Frixel. Le framework Phoenix est très complet, innovant, et bien conçu. La séparation des responsabilités est très claire (contexts, schemas, controllers, LiveViews), les outils intégrés sont très riches, et LiveView permet de construire des interfaces réactives côté serveur sans avoir à maintenir un framework JavaScript séparé.

Côté données, Phoenix fournit l'ORM Ecto, conçu pour fonctionner par défaut avec PostgreSQL, avec un système de migrations versionné et les changesets qui permettent la validation des données avant toute écriture en base.

`mix phx.gen.auth` nous fournit un système d'authentification poussé en une commande, avec stockage des mots de passe hashés avec bcrypt en base, ainsi

que le magic link, un système de vérification de compte et de réinitialisation de mot de passe grâce à un envoi de token par email.

Les tests s'appuient sur ExUnit pour les tests unitaires, ConnCase pour les routes HTTP et Phoenix.LiveViewTest pour les interactions LiveView.

L'intégration continue est assurée par GitHub Actions, dont le pipeline exécute les tâches d'installation et de validation fournies par l'écosystème Phoenix : `mix format --check-formatted` et `mix credo --strict` pour le linting et le formatage, `mix test` pour l'exécution des tests et `mix compile --warnings-as-errors` pour la compilation stricte du projet.

2.3 Architecture générale

Lifequest repose sur une architecture en trois couches strictement séparées, afin de respecter la convention proposée par le framework.

Couche présentation : HEx est un langage de templating permettant de construire nos pages webs en Elixir. Ce sont les LiveViews qui sont chargées d'interpréter ce code. Lors de leur création, un état est généré côté serveur et une Websocket persistante est ouverte pour communiquer avec le navigateur. Les interactions utilisateur déclenchent des événements traités par le serveur qui renvoie le diff partiel du DOM, permettant de changer l'interface sans rechargement de la page.

Couche métier : Ce sont les contextes Elixir qui portent ces responsabilités. Chaque contexte expose des fonctions publiques qui pourront être appelées par l'ensemble des LiveView. Ces dernières doivent passer par les fonctions des contextes pour interagir avec la base de données. Cela centralise la logique métier, et facilite les tests.

Couche données : Les schémas Ecto définissent la structure des données et les changesets qui valident les entrées avant toute écriture. Ecto génère des requêtes paramétrées afin d'éviter les injections SQL.

Une autre fonctionnalité offerte par Phoenix s'appelle Scope. Il s'agit d'une struct représentant l'utilisateur actuel qui est passée en premier argument à toutes les fonctions de contexte. Cela garantit l'isolation des données entre les utilisateurs,

car elle est utilisée comme filtre dans toutes les requêtes Ecto. On peut également s'appuyer sur elle pour la gestion des droits, par exemple entre admin et user, ou entre compte free ou premium.

Pour résumer, à l'appel d'une route : **Voir Annexe 1.1 — schéma UML de la séquence de traitement d'une requête**

- la requête HTTP initiale déclenche le `mount/3` de la LiveView
- les fonctions de contextes nécessaires au rendu initial sont appelées
- les requêtes SQL sont envoyées à PostgreSQL grâce à Ecto
- les données sont placées en `assigns` sur la socket et stockées en mémoire vive
- le template HEx s'affiche, en utilisant ces données

Par la suite, les interactions utilisateurs déclencheront des fonctions `handle_event/3` qui fonctionnent de manière similaire, et qui permettront de modifier le DOM sans rechargement ou d'émettre des messages PubSub pour avertir d'autres LiveViews abonnées de la mise à jour de leur état en temps réel grâce à `handle_info/3`

3. CP1 — Installer et configurer son environnement de travail en fonction du projet

IDE et extensions

Pour le développement, j'ai utilisé VSCode car c'est l'IDE que j'ai l'habitude d'utiliser. Son catalogue d'extensions m'a permis d'installer des aides au développement adaptées à la stack : ElixirLS pour l'autocomplétion et la navigation dans le code Elixir, l'extension PlantUML pour la création et la visualisation de mes schémas UML directement dans l'éditeur, et TODO Highlight pour garder en tête les tâches à effectuer dans le code.

Outillage Mix

Le projet repose principalement sur Elixir et Mix, son gestionnaire de tâches natif. J'ai créé une tâche personnalisée, `mix precommit`, afin de grouper les commandes de formatage, de lint, de build et d'exécution des tests et de m'assurer

de la qualité de mon code avant de le pousser sur la branche distante. Toutes ces étapes sont identiques à celles exécutées sur la pipeline CI à la création d'une pull request, ce qui me permet de capturer les problèmes plus tôt dans mon workflow.

`mix setup` permet d'installer le projet et ses dépendances, créer les bases de données et jouer leurs migrations. On dispose de deux bases de données par défaut : la base de développement et la base de tests, permettant d'effectuer des tests reproductibles sur cette dernière sans affecter les données de développement.

Commandes de lancement

`mix phx.server` permet de lancer l'application en mode développement sur le port 4000 de la machine hôte. `iex -S mix phx.server` lance en parallèle un shell interactif, utile pour tester des expressions Elixir en contexte d'application.

Même si j'ai travaillé seul sur ce projet, j'ai également utilisé les outils de versioning Jira, Git, Github, de CI GitHub Actions de conteneurisation Docker et d'orchestration. Mon utilisation de ces outils seront détaillés dans les parties de ce rapport qui leurs sont consacrées.

4. CP2 — Développer des interfaces utilisateur

4.1 Conception des interfaces avec Phoenix LiveView

Lifequest repose entièrement sur Phoenix LiveView pour la construction de ses interfaces. Ce modèle se distingue des approches frontend classiques (React, Vue) en maintenant l'état de l'interface côté serveur : le navigateur établit une connexion WebSocket persistante avec le serveur, qui calcule les changements de DOM et envoie uniquement les diffs nécessaires au client. Cette architecture élimine le besoin d'un framework JavaScript séparé et d'une API entre le front et le back, tout en offrant une réactivité comparable à celle d'une Single Page Application.

Chaque LiveView suit une structure définie par trois callbacks principaux :

- `mount/3` initialise l'état de la page en chargeant les données nécessaires dans les assigns.

- `handle_event/3` traite les interactions utilisateur comme les clics ou les soumissions de formulaire et met à jour l'état en conséquence.
- `handle_info/3` reçoit les messages asynchrones, notamment ceux émis via PubSub.

Le template HEx associé reflète automatiquement chaque changement d'état sans rechargement de page.

Il est possible de factoriser les composants réutilisables dans des modules personnalisés, pour permettre une organisation propre à chaque équipe.

L'implémentation de base de Phoenix les regroupe dans `core_components.ex`, qui expose des composants fonctionnels tels que `<.input>`, `<.form>` et `<.icon>`. Cette centralisation garantit la cohérence visuelle et comportementale à travers l'application : un champ de formulaire se comporte et s'affiche de manière identique partout, et toute modification s'applique en un seul point.

Exemple concret de la LiveView `dashboard_live` : Voir annexe 2.1

A l'initialisation de la LiveView, la fonction `mount/3` est appelée, et appelle elle-même les différentes fonctions du contexte permettant de récupérer les données en base permettant l'affichage des données sur le dashboard, avant de les placer dans les assigns. Parmi elles, la fonction `load_dashboard_data` qui vient appeler différentes fonctions du contexte `Finances` pour récupérer les revenus et les dépenses de l'utilisateur en base. Cela permet de générer les deux graphiques. Si des revenus ou des dépenses récurrentes ont été générées automatiquement en base sans avoir été validées, elles apparaissent dans une section de la page, `pending_recurring_section`, permettant à l'utilisateur de les valider une par une. En cliquant sur le bouton valider, l'évènement `validate_recurring` est déclenché et capté par un `handle_event` qui va lui-même essayer d'appeler la fonction `validate_recurring/3` du contexte `Finances` pour mettre à jour les informations dans les assigns de la socket. La transaction perd son état pending, elle disparaît donc de la section dédiée à ces dernières.

4.2 Charte graphique et responsive design (Tailwind CSS / daisyUI)

Pour la couche de présentation, j'ai utilisé Tailwind CSS v4 associé à daisyUI, qui sont les outils fournis par Phoenix et que j'utilise au quotidien chez Frixel.

Tailwind est un framework CSS utility-first : plutôt que de définir des classes sémantiques, on compose les styles directement dans le HTML grâce à des classes atomiques (`flex` , `gap-4` , `rounded-lg` ...). Cela évite d'écrire du CSS custom et élimine les conflits de nommage.

A la compilation, seules les classes effectivement utilisées sont incluses dans le bundle final, ce qui allège le poids des assets en production.

daisyUI vient compléter Tailwind en proposant des composants prêts à l'emploi : `btn` , `card` , `badge` , `menu` ... Ce sont de simples classes CSS qui regroupent des combinaisons de classes Tailwind, sans JavaScript embarqué.

Le système de thèmes est configuré dans `app.css` où l'on définit nos couleurs `primary` , `base-100` , `base-200` ... Changer le thème revient à changer la valeur de ces variables, sans toucher aux composants. Lifequest propose trois modes : clair, sombre, et système. Ce dernier détecte automatiquement la préférence du navigateur.

La bascule est gérée côté client via l'attribut `data-theme` sur la balise `html` , ce qui évite un rechargement de page.

Pour le responsive, j'ai utilisé les préfixes de breakpoints de Tailwind (`sm:` , `md:` , `lg:`), qui permettent de modifier les valeurs voulues selon la taille de l'écran inline, sans avoir à écrire de media-queries.

4.3 Accessibilité (RGAA)

Pour travailler sur l'accessibilité, je me suis basé sur les retours d'audit de Lighthouse sur chacune des pages de l'application, ce qui m'a amené à étudier les attributs aria manquants et les contrastes de couleur.

Les composants daisyUI fournissent des labels aria de base sur les éléments courants. J'ai ajouté les labels manquants sur les boutons sans texte.

De plus, sur certains éléments qui se répètent sur une même page, j'ai ajouté un label dynamique. Par exemple sur les icônes de modification et de suppression des différentes transactions de la page "Finances" j'ai inclu le libellé de la transaction ciblée, ce qui donne des labels comme "Modifier Loyer" ou "Supprimer Salaire net", permettant à l'utilisateur de distinguer chaque bouton sans ambiguïté.

Toutes les pages de l'application ont un score d'accessibilité supérieur à 90 suite aux audits Lighthouse.

Certaines interactions complexes comme les graphiques n'ont pas fait l'objet de tests spécifiques avec des lecteurs d'écran.

4.4 Conformité RGPD — Mentions légales

Lifequest collecte et traite des données personnelles, ce qui implique de respecter le Règlement Général sur la Protection des Données. J'ai créé une page dédiée, accessible depuis le footer sur l'ensemble de l'application, y compris sans être connecté.

La base légale retenue est l'exécution d'un contrat au sens de l'article 6(1)(b) du RGPD : en créant un compte, l'utilisateur accepte que ses données soient traitées pour lui permettre d'utiliser le service. Les données sont conservées pendant toute la durée d'activité du compte, puis supprimées à sa résiliation. Les tokens d'authentification expirent automatiquement après 60 jours.

La page liste les droits de l'utilisateur : accès, rectification, effacement, portabilité et opposition. Le droit à l'effacement est directement accessible depuis les paramètres du compte. La suppression du compte entraîne la suppression en cascade de toutes les données associées en base.

4.5 Tests des composants d'interface

Les interfaces de Lifequest sont testées via le module `Phoenix.LiveViewTest` qui simule le cycle de vie complet d'une LiveView dans un environnement de test.

Les tests utilisent le module `ConnCase` pour établir une connexion HTTP simulée avec une session authentifiée.

La stratégie de test porte sur trois aspects : la vérification du rendu HTML attendu, la présence des éléments d'interface critiques et le comportement en réponse aux interactions utilisateur.

`describe` permet de grouper des tests unitaires qui partagent la même situation initiale `test` est un test unitaire, qui contient nos assertions `assert html =~ "texte attendu"` permet de vérifier la présence d'un contenu textuel dans le rendu `assert has_element?(live_view, "sélecteur CSS")` permet de vérifier la présence d'un élément spécifique dans le DOM.

Exemple concret des tests de la LiveView `dashboard_live` : Voir annexe 2.1

`setup :register_and_log_in_user` permet de simuler la situation initiale : un utilisateur est connecté. C'est nécessaire car toutes les opérations testées ne sont déclenchables que dans cette situation précise.

`create_recurring_last_month(scope, attrs \\ %{})` est une fonction qui permettra de créer une opération récurrente au besoin. Des valeurs par défaut sont définies, mais peuvent être écrasées en fournissant le deuxième argument facultatif.

`test "shows pending recurring incomes from last month"` crée une opération récurrente à valider par défaut et s'assure qu'elle s'affiche bien sur la page

`test "does not show recurring already validated this month"` crée une opération récurrente et insère sa validation en base pour le mois courant, puis vérifie qu'elle n'apparaît bien pas dans la liste des revenus à valider

`test "validate_recurring duplicates transaction to current month"` simule la validation manuelle par l'utilisateur, et s'assure que cela entraîne la disparition de la ligne sur la page

4.6 Sécurité des interfaces (XSS, CSRF)

Phoenix fournit des protections contre les principales vulnérabilités web directement dans sa couche de templating. Contre le Cross-Site Scripting (XSS), HEEx échappe automatiquement toute variable interpolée via `{@assign}` : si une

donnée utilisateur contient du HTML ou du JavaScript, le contenu est affiché tel quel, sans être interprété par le navigateur.

Aucune balise `<script>` inline n'est présente dans les templates HEEEx de Lifequest : l'ensemble du JavaScript est isolé dans le répertoire `assets/js/`.

Contre les attaques Cross-Site Request Forgery (CSRF), Phoenix génère un token unique intégré à chaque formulaire via le composant `<.form>`. Ce token est vérifié côté serveur à chaque soumission, ce qui garantit que la requête provient bien de l'application et non d'un site tiers.

Les Scopes et leur présence dans les arguments des fonctions de Repo garantissent que l'utilisateur actif peut récupérer les données lui appartenant, et seulement celles-ci.

5. CP3 — Développer des composants métier

5.1 Architecture en contextes (Finances, Accounts)

Les contextes sont un pilier d'organisation du code de Phoenix. Ils regroupent la logique métier dans des modules, qui exposent une API publique que la couche présentation peut utiliser. Dans mon application, deux contextes ont été mis en place, `Accounts` qui gère l'authentification et le Scope, et `Finances` qui contient la logique métier de l'application.

Ce dernier expose notamment les fonctions CRUD générées à la création du schéma ainsi que toutes les fonctions d'accès aux données plus spécifiques. C'est le seul moyen pour les LiveViews de communiquer avec l'ORM, elle n'appellent jamais `Repo` directement, et j'ai appliqué cette contrainte strictement dans l'ensemble du projet.

Les `Scopes`, apportés par Phoenix 1.8 et générés par `phx.gen.auth` permettent d'isoler les données de manière sécurisée. L'utilisateur actif est passé en premier argument de toutes les fonctions de contexte, et est utilisé pour filtrer les requêtes Ecto. De cette façon, on est certain qu'un utilisateur ne peut jamais accéder aux données d'un autre.

5.2 Développement défensif et gestion des erreurs

Pour présenter cette partie, laissez moi vous parler du concept de pattern matching. Il s'agit d'une fonctionnalité Elixir qui permet à des fonctions qui portent le même nom d'avoir des corps différents selon leur arité ou selon la forme des arguments reçus. Quand une fonction avec plusieurs définitions est appelée, Elixir essaye de pattern match avec chacune d'entre elle dans l'ordre du fichier et exécute la première qui correspond. Il est très commun de terminer la liste d'implémentation d'une fonction par une "catch all", qui permet d'avoir un comportement par défaut si aucune des implémentations spécifiques ne correspond.

Ecto s'appuie sur cette mécanique pour générer ses valeurs de retour grâce à des objets appelés `changesets`. Ils sont utilisés pour s'assurer de la validité des données côté serveur. Si une validation échoue, l'écriture en base est annulée et un tuple `{:error, _changeset}` est retourné, et le `changeset` contient les messages d'erreur, permettant de les afficher directement sur le formulaire sans recharger la page ni crash.

5.2.a Exemples concrets de pattern matching : *Voir annexe 3.1*

La fonction `format_transaction_type/1` du Dashboard a deux clauses, l'une qui matche sur `%{direction: :income}` et l'autre sur `%{direction: :expense}`. Dans les deux cas, l'objet passé en argument doit être un objet qui contient ces deux types, en l'occurrence une transaction. Appeler cette fonction avec une transaction revenu ou une transaction dépense déclenche automatiquement la bonne clause, sans `if` ni `cond`.

La fonction `format_income_type/1` prend le relais pour les revenus, et pattern match sur le type de transaction pour renvoyer une chaîne de caractère à afficher dans le DOM. La clause catch all, représentée par le symbole `_` gère les cas qu'on aurait pas prévu sans faire crasher l'application.

Ce principe de pattern matching permet aussi d'adapter le comportement de la fonction dépendamment de la valeur de retour d'un appel de fonction présent dans son corps. Cet `handle_event/3` lance un case pour appeler `Finances.validate_recurring/3`, et pattern match sur `{:ok, }` et `{:error, }` pour

effectuer la suite de ses opérations. Les deux cas sont traités explicitement, et aucun chemin d'exécution n'est ignoré silencieusement. C'est un pattern extrêmement commun en Elixir.

5.3 Communication en temps réel avec Phoenix PubSub

Phoenix PubSub permet à des processus de s'abonner à des canaux nommés et de recevoir des messages asynchrones émis par d'autres processus. Dans Lifequest, je l'utilise pour propager les mises à jour de données entre LiveViews. L'abonnement se fait dans `mount/3` au moment de la connexion à la socket. Les canaux sont scopés par utilisateur, avec un nom du type

`"user:#{user_id}:transactions"`. Un message publié sur ce canal n'est reçu que par les LiveViews de cet utilisateur précis. Cela garantit que les événements d'un utilisateur ne se propagent pas aux sessions d'un autre.

Quand un message PubSub est reçu, c'est le callback `handle_info/2` qui le traite. Il fonctionne de la même manière qu'un `handle_event/3`, mais déclenché par un message interne plutôt que par une action utilisateur.

5.4 Tests unitaires des composants métier

La séparation entre contexte et LiveView a un avantage direct pour les tests : on peut tester entièrement la logique métier de `Finances` sans instancier de LiveView ni simuler de connexion HTTP.

Les contextes `Finances` et `Accounts` sont testés dans `test/lifequest/finances_test.exs` et `test/lifequest/accounts_test.exs` en suivant la convention Phoenix. Chaque fichier est structuré de la même façon : chaque fonction est décrite par un bloc `describe` qui contient le cas nominal, les cas d'erreur, et le test d'isolation par `Scope`.

Ce dernier est systématique. Pour chaque fonction qui récupère des données en base, on crée un second utilisateur et s'assure que l'appel de fonction ne retourne pas de données appartenant au premier. Ce test permet de garantir que les données ne peuvent pas fuiter entre les utilisateurs. **Voir annexe 3.2**

Pour créer facilement des données de tests sans dupliquer de code, Phoenix préconise de créer des fonctions helpers appelées fixtures. Ce sont des fonctions

qu'on appelle dans chacun de nos tests pour définir des conditions initiales déterminées, avec des valeurs par défaut que l'on peut écraser selon les besoins du test. Cette approche rend les tests très lisibles, s'assure de leur reproductibilité et permet de se concentrer sur ce qu'ils vérifient.

5.5 Sécurité des composants métier

La sécurité des composants métier repose sur plusieurs couches complémentaires.

Les changesets Ecto sont construits à partir de formulaires, dont les champs sont transmis à une fonction `cast/3`. Cependant, les champs système comme le `user_id` ne passent jamais par cette fonction, et sont ajoutés automatiquement après la validation à partir du Scope contenu dans la socket. Cela empêche tout utilisateur malveillant de les manipuler pour accéder à des données qui ne leur appartiennent pas.

Les changesets incluent aussi des validations côté serveur systématiques : `validate_required` pour les champs obligatoires, `validate_number` pour les montants... On sait que la vérification côté client est utile pour l'expérience utilisateur mais n'est pas sécurisée, il est donc nécessaire de conserver cette étape. Il s'agit de notre source de vérité et de notre garantie que les données insérées en base sont toujours valides.

Les requêtes Ecto sont systématiquement filtrées grâce au Scope. On ne récupère jamais une donnée par son seul identifiant, on filtre toujours la requête par l'id de l'utilisateur grâce au pattern `where: t.account_id in ^account_ids`

On ne récupère jamais une transaction par son seul identifiant : la requête joint toujours l'identifiant du compte, soit en vérifiant que c'est bien la clé étrangère de la ligne, soit en faisant une jointure sur la table users si nécessaire.

Enfin, certaines données sont définies comme des Enum, ce qui permet de s'assurer que le contenu de la donnée appartient à une liste prédéfinie.

6. CP4 — Contribuer à la gestion d'un projet informatique

6.1 Méthode de développement itérative

Même si j'ai travaillé seul sur ce projet, j'ai appliqué les bonnes pratiques utilisées au quotidien chez Frixel afin de maintenir un historique propre, traçable et permettant le travail en équipe.

Chaque itération suit le même cycle : création du ticket Jira, ouverture d'une branche Git, développement, écriture des tests, création d'une Pull Request, validation par la CI, merge sur `main`.

L'application est restée fonctionnelle à chaque étape de développement, pratique importante si l'on souhaite la mettre à jour régulièrement sans avoir à arrêter l'application en production.

6.2 Planification et suivi des tâches (Jira)

Chaque fonctionnalité commence par un ticket Jira qui décrit le besoin et les critères d'acceptation, ce qui m'oblige à définir clairement ce que je veux produire avant de coder. Le numéro du ticket est ensuite porté par la branche Git, les commits et la Pull Request, ce qui rend l'historique entièrement traçable.

Le workflow Jira suit trois colonnes : backlog, en cours, terminé. Simple, mais suffisant pour garder une vision claire de l'avancement et prioriser les prochaines itérations.

6.3 Outils collaboratifs (GitHub, conventions de commits)

La suite du processus est la création d'une branche GIT selon la convention de nommage établie chez Frixel. Cette convention permet de relier directement le ticket, la branche et les merges sur main, eux même nommés en suivant une convention similaire.

`LQ-XX type(description)` :

- `LQ-XX` correspond au numéro de ticket JIRA

- `type` correspond à la nature de la modification (feat, fix, refactor...)
- `description` est un court résumé des changements apportés par la mise à jour

Une fois les modifications terminées, je push mon code sur une nouvelle branche distante sur GitHub puis je crée une Pull Request vers `main` ce qui déclenche automatiquement le pipeline CI grâce à GitHub Actions.

6.4 Qualité du code (Credo, mix format, CI)

Lorsqu'une PR est ouverte, une CI est automatiquement exécutée grâce à Github Actions.

Cette pipeline exécute trois vérifications primordiales : le formatage du code avec `mix format`, le lint avec `mix credo` qui applique des règles de lisibilité et de complexité définies en amont, l'exécution des tests avec `mix test`.

Ces trois vérifications sont enchaînées dans une tâche custom `mix precommit`, que j'exécute avant chaque commit, me permettant de détecter les problèmes avant de push mon code en distant.

Toutes ces étapes permettent de garantir de conserver un historique et une branche `main` propre, où chaque commit est validé en amont.

7. CP5 — Analyser les besoins et maquetter une application

7.1 Analyse du cahier des charges et identification des besoins

Comme expliqué en introduction, le point de départ de Lifequest est un constat personnel : aucune application existante ne permet à la fois de suivre ses flux financiers mensuels et de simuler, planifier et concrétiser son avenir financier en fonction d'objectifs de vie concrets.

À partir de ce problème, j'ai identifié les besoins fonctionnels principaux :

- Saisie des revenus et dépenses par catégorie, avec support des transactions récurrentes

- Tableau de bord visuel synthétisant la situation du mois courant
- Projections financières à horizon personnalisable, basées sur les données déclarées
- Simulation de placements avec différents niveaux de risque
- Suivi d'objectifs financiers définis par l'utilisateur

Certaines fonctionnalités ont été volontairement mises hors périmètre pour garder le projet réaliste : le support multi-devises, la connexion à des comptes bancaires réels via API open banking et à des produits financiers réels pour faire de vrais placements.

7.2 User stories

Les user stories ont servi de fil directeur pour prioriser le développement. En voici quelques-unes représentatives :

En tant qu'utilisateur, je veux renseigner ma situation financière afin que l'application puisse calculer des projections réalistes. Critères : le formulaire accepte un montant d'épargne actuelle, de dettes et de revenu mensuel net. Les données sont sauvegardées et réutilisées dans toutes les projections.

En tant qu'utilisateur, je veux saisir mes revenus et dépenses par catégorie afin de connaître précisément la répartition de mes flux mensuels. Critères : je peux créer une transaction en choisissant un type (salaire, loyer, loisirs...), un montant et une date. Les transactions peuvent être marquées comme récurrentes.

En tant qu'utilisateur, je veux visualiser la répartition de mes dépenses sous forme de graphique afin d'identifier les postes les plus importants d'un coup d'oeil. Critères : le dashboard affiche deux donuts SVG, l'un pour les revenus et l'autre pour les dépenses, avec les montants par catégorie.

En tant qu'utilisateur, je veux simuler un placement mensuel sur plusieurs années afin de savoir combien j'aurai accumulé selon différents niveaux de rendement. Critères : je saisis un montant mensuel et un horizon en années. L'application affiche les projections pour trois scénarios : prudent, modéré et dynamique.

En tant qu'utilisateur, je veux supprimer mon compte et toutes mes données afin de pouvoir exercer mon droit à l'effacement. Critères : une action dans les

paramètres supprime le compte et toutes les données associées en cascade, sans possibilité de récupération.

7.3 Maquettes et enchaînement des écrans

Les maquettes ont été réalisées en amont grâce à l'outil Figma. Elles sont inspirées des différentes applications que j'ai été amené à utiliser dans mon parcours, ou à étudier au début de mon étude de marché.

Elles couvrent les différents écrans de l'application, en suivant le parcours utilisateur classique, même s'il est très simple. En dehors du parcours d'inscription/connexion, tous les écrans sont traités au même niveau et accessibles depuis le menu situé dans la barre latérale de navigation.

La seule exception étant les formulaires que les utilisateurs peuvent utiliser pour remplir leurs informations financières, qui sont accessible par le clic de boutons situés dans la page finances. Ces formulaires sont simplifiés au maximum pour faciliter le parcours utilisateur et limiter les erreurs humaines.

L'utilisation de donuts SVG pour l'affichage des informations est directement inspiré d'applications concurrentes, permettant à l'utilisateur de comprendre la répartition de ses revenus et dépenses et sa situation financière en un coup d'oeil.

8. CP6 — Définir l'architecture logicielle d'une application

8.1 Architecture multicouche (web / contextes / schémas)

Comme indiqué plus tôt dans ce rapport, l'architecture de Lifequest suit la convention Phoenix de découplage des couches MVC. La couche Modèle contient les schémas Ecto et PostgreSQL, la couche Vue contient les LiveViews et les templates HEx, et la couche métier qui fait le lien entre ces éléments, grâce aux fonctions de contextes exposées dans `Finances` et `Accounts`.

Les dépendances sont unidirectionnelles : la couche web dépend des contextes, les contextes dépendent des schémas, les schémas dépendent de la base. Une

LiveView ne connaît pas la structure d'une table, elle appelle une fonction du contexte qui lui retourne des données.

Cette approche apporte des avantages concrets de testabilité, de maintenance, de lisibilité, de factorisation, et permettent le travail en collaboration en évitant les conflits.

8.2 Choix du framework et de l'ORM (Phoenix / Ecto)

Ce choix s'est imposé naturellement, car c'est la stack sur laquelle je travaille en entreprise. Laissez moi tout de même vous présenter d'autres avantages concrets, au delà de la familiarité.

LiveView élimine le besoin d'un framework front et d'une API le reliant au Back, et est optimisé pour la conception d'interface réactives en temps réel, ce qui est primordial pour l'expérience utilisateur dans le cadre d'applications modernes.

Elixir s'exécute sur la BEAM, la machine virtuelle d'Erlang, qui gère la concurrence par des processus légers et isolés. Chaque utilisateur connecté dispose de son propre processus LiveView sans que sa charge n'affecte les autres.

Ecto est l'ORM de référence de l'écosystème Elixir. Il se distingue par l'absence de magie implicite : les requêtes sont composées explicitement avec `Ecto.Query`, et les données passent toujours par un changeset avant d'être écrites.

PostgreSQL, grâce à son support natif des types `decimal` pour les montants (les `float` introduisent des erreurs d'arrondi) et ses types `enum` natifs qui permettent de contraindre les valeurs directement au niveau de la base a été retenu pour sa robustesse sur les données financières,

Le serveur HTTP utilisé est Bandit, le successeur moderne de Cowboy, depuis la version 1.7 de Phoenix.

8.3 Stratégie de sécurité par couche

La sécurité est traitée à chaque couche de l'application garantissant une protection robuste et limitant les attaques possibles au maximum.

Couche Présentation : Les templates HEx échappent automatiquement toutes les variables interpolées, ce qui élimine les risques XSS par défaut. Le composant `<.form>` injecte un token CSRF dans chaque formulaire vérifié côté serveur à chaque soumission. Les routes protégées sont regroupées dans un bloc `live_session :require_authenticated_user` qui interdit l'accès sans session valide.

Couche métier : les changesets valident toutes les données avant toute écriture. Les champs système ne passent jamais par `cast/3` et sont assignés programmatiquement. Toutes les requêtes filtrent par `current_scope`, ce qui rend impossible l'accès aux données d'un autre utilisateur même en manipulant un identifiant.

Couche données : Ecto génère des requêtes paramétrées : les valeurs sont transmises séparément de la requête SQL, ce qui protège nativement contre les injections. Les enums PostgreSQL garantissent l'intégrité des valeurs sans code applicatif supplémentaire.

Côté authentification : les magic links sont des tokens à usage unique stockés sous forme hashée en base. Le token brut n'est jamais persisté, et chaque token expire après 60 jours.

8.4 Éco-conception

Cette application n'a pas particulièrement conçue avec l'écologie en tête, mais le choix de cette stack, son origine historique et les optimisations qui en découlent permettent de limiter notre consommation d'espace serveur, de puissance de calcul nécessaire et de réseau, limitant par conséquent nos besoins en électricité.

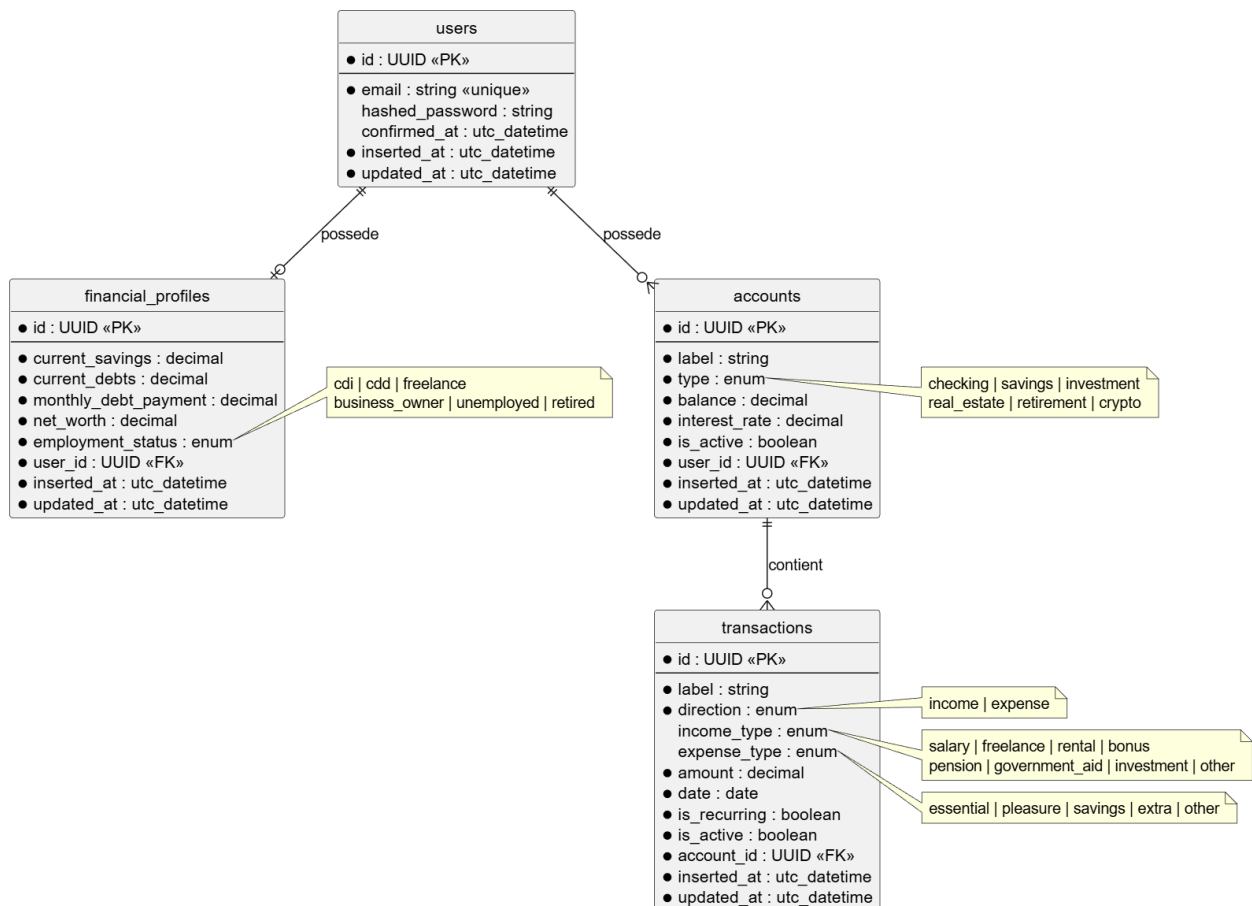
Elixir est un langage compilé. Pour la mise en production, un binaire contenant le strict nécessaire au fonctionnement de l'application est généré : code minifié, dépendances compilées, assets bundlés...

Il repose sur la BEAM, la machine virtuelle sous-jacente, qui a été conçue à l'origine pour les systèmes de télécommunications où la densité de connexions simultanées est élevée. Elle gère la concurrence par des processus légers, dont chacun n'occupe que quelques kilo-octets en mémoire au démarrage.

LiveView contribue à réduire la consommation réseau par sa conception même : le serveur calcule le diff du DOM et n'envoie via WebSocket que les parties modifiées. Chaque utilisateur maintient une seule connexion persistante, en lieu et place des multiples requêtes HTTP qu'une architecture REST classique générerait à chaque interaction.

9. CP7 — Concevoir et mettre en place une base de données relationnelle

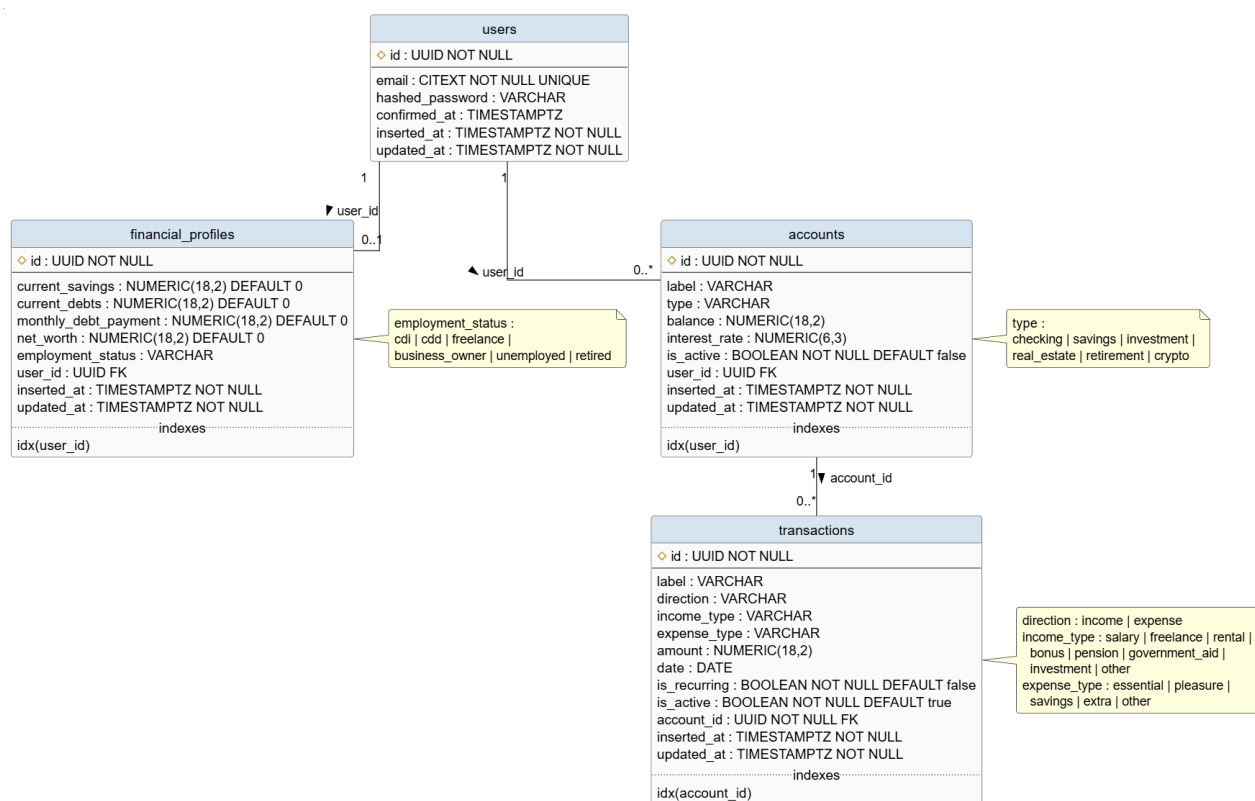
9.1 Modèle Conceptuel de Données (MCD)



Le modèle de données de Lifequest repose sur quatre entités : `users` , `financial_profiles` , `accounts` et `transactions` . Les relations sont les suivantes : un utilisateur possède zéro ou un profil financier, zéro ou plusieurs comptes, et chaque compte contient zéro ou plusieurs transactions.

Le choix de l'entité `Transaction` unifiée mérite une explication. Au démarrage du projet, j'avais envisagé deux tables séparées pour les revenus et les dépenses. J'ai rapidement constaté que cette séparation rendait les requêtes d'agrégation complexes : calculer un solde mensuel net impliquait de joindre deux tables de structures différentes. J'ai choisi de fusionner les deux en une seule entité avec un champ `direction` (`:income` ou `:expense`), et des champs `income_type` et `expense_type` optionnels selon la direction. Cette décision est développée plus en détail en section 15.1.

9.2 Modèle Logique de Données (MLD)



Les quatre tables PostgreSQL correspondent directement aux entités du MCD. Quelques choix techniques méritent d'être soulignés.

Les clés primaires sont des UUID (`binary_id` chez Ecto) plutôt que des entiers auto-incrémentés. Ce choix évite l'exposition d'un compteur prédictible dans les URLs, ce qui rend plus difficile l'énumération des ressources.

Les montants utilisent le type `NUMERIC(18, 2)` plutôt que `FLOAT`. En arithmétique flottante, `0.1 + 0.2` ne vaut pas exactement `0.3`. Ce

comportement est inacceptable pour une application financière : on utilise donc des décimaux à précision fixe.

Les champs de catégorie (`direction` , `income_type` , `expense_type` , `employment_status` , `type`) sont stockés en `VARCHAR` et validés côté application par les changesets Ecto, avec des enums définis au niveau du schéma Elixir. Les colonnes fréquemment filtrées sont indexées : `user_id` sur `financial_profiles` et `accounts` , `account_id` sur `transactions` .

9.3 Migrations Ecto et scripts de création

Ecto impose un processus strict pour les modifications de schéma : `mix ecto.gen.migration nom_migration` génère un fichier horodaté dans `priv/repo/migrations/` , que l'on édite puis applique avec `mix ecto.migrate` . Les migrations sont versionnées avec le reste du code, ce qui permet de rejouer l'historique complet du schéma sur n'importe quel environnement.

La convention est d'utiliser `change/0` qui rend la migration réversible automatiquement via `mix ecto.rollback` . On ne recourt à `up/do` et `down/0` que pour les rollbacks qui nécessitent une logique custom.

L'historique des migrations illustre bien l'évolution du schéma. La table `transactions` a d'abord été créée avec une clé étrangère directe vers `users` . Une migration ultérieure a supprimé ce lien pour le remplacer par une clé vers `accounts` , ce qui correspond à la décision d'architecture de passer par les comptes bancaires comme point d'entrée des transactions. Une règle stricte s'applique : on ne modifie jamais une migration déjà fusionnée sur `main` . Toute correction passe par une nouvelle migration.

9.4 Sécurité et droits d'accès

L'isolation des données est garantie par la chaîne de clés étrangères : une `Transaction` appartient à un `Account` , qui appartient à un `User` . On ne récupère jamais une transaction par son seul identifiant, on filtre toujours via le scope de l'utilisateur, ce qui garantit qu'un accès par un ID manipulé retourne une erreur 404 plutôt qu'une fuite de données.

L'utilisateur PostgreSQL utilisé par l'application dispose uniquement des droits nécessaires : lecture et écriture sur les tables applicatives, sans droits d'administration. Il ne peut pas modifier le schéma, créer des tables ou accéder aux tables système.

9.5 Jeu d'essai

Le fichier `priv/repo/seeds.exs` insère un jeu de données représentatif : un utilisateur avec un profil financier renseigné, plusieurs comptes de types différents (courant, épargne, investissement), et des transactions réparties sur plusieurs mois avec des directions et catégories variées.

Les seeds sont idempotents : ils utilisent `on_conflict: :nothing` pour ne pas créer de doublons si on les rejoue. Parmi les cas limites couverts : un utilisateur sans profil financier, une catégorie sans transaction associée, une transaction inactive et un montant à zéro.

10. CP8 — Développer des composants d'accès aux données SQL et NoSQL

10.1 Requêtes Ecto (CRUD sécurisé)

- Les 4 opérations exposées par le contexte `Finances` :
`list_transactions/1`, `get_transaction!/2`, `create_transaction/2`, `delete_transaction/2`
- Requête d'agrégation : groupement par type de transaction sur le mois courant pour les donuts du dashboard — montrer la query Ecto composée
- Requête avec filtre temporel : sélectionner les transactions du mois courant via `where: fragment("date_trunc('month', ?) = date_trunc('month', now())", t.date)`
- `Ecto.Query` composable : clauses `where` ajoutées conditionnellement (ex : filtrage par catégorie)

10.2 Gestion de l'intégrité et des conflits d'accès (transactions)

- `Repo.transaction/1` pour les opérations atomiques : si la création d'un profil échoue, le compte associé n'est pas créé non plus
- Rollback automatique en cas d'erreur dans le bloc transaction
- Gestion des conflits `on_conflict` : utile pour les seeds et les opérations idempotentes
- Pattern `with` pour chaîner plusieurs opérations avec gestion d'erreur propre

10.3 Validation et contrôle des entrées côté serveur

- Changeset comme couche de validation : `cast/3` → `validate_required/2` → `validate_number/3` → `validate_inclusion/3`
- Validation côté serveur uniquement : jamais faire confiance aux données du client, même si un contrôle existe en frontend
- Les erreurs de changeset remontent automatiquement vers le formulaire LiveView via `{:error, changeset}` → `to_form(changeset)`
- Montrer le changeset `Transaction.changeset/2` commenté comme exemple représentatif

10.4 Tests unitaires et de sécurité des accès aux données

- Tests du contexte `Finances` : vérifier que `list_transactions(scope)` retourne uniquement les transactions de ce scope
- Test d'isolation : créer des transactions pour `other_scope`, vérifier qu'elles sont absentes du résultat de `scope`
- Test des erreurs : un changeset avec des données invalides retourne `{:error, changeset}` avec les bons messages

10.5 Protection contre l'injection SQL

- Ecto utilise des requêtes paramétrées nativement : le `^` dans les queries (`where: t.id == ^id`) passe par un paramètre PostgreSQL (`$1`), jamais par interpolation de chaîne
- Pas de requêtes SQL raw sauf cas exceptionnel avec `Ecto.Adapters.SQL.query/3` et paramètres explicites

- Exemple de ce qu'Elixir génère : `SELECT * FROM transactions WHERE id = $1` — le paramètre est transmis séparément
-

11. CP9 — Préparer et exécuter les plans de tests d'une application

11.1 Plan de tests

⚠ A CRÉER avant le jury : aucun plan de tests formel (tableau structuré) n'existe. Produire le tableau ID/fonctionnalité/précondition/action/résultat attendu/statut et le déposer en Annexe C. Les tests ExUnit existants sont la source de vérité pour remplir ce tableau.

- Objectif du plan : valider que chaque fonctionnalité fonctionne correctement et que les données sont strictement isolées par utilisateur
- Périmètre couvert : contextes `Finances` et `Accounts` (tests unitaires) + toutes les LiveViews (tests d'intégration)
- Types de tests retenus : unitaires (contextes), intégration (LiveView avec `ConnCase`), sécurité (isolation par scope, redirections)
- Critères de réussite : 0 tests en échec, 0 warning de compilation, CI verte sur chaque PR
- Insérer le plan de tests complet en Annexe C

11.2 Tests unitaires (ExUnit)

- Framework : ExUnit intégré à Elixir, pas de dépendance externe
- Structure standard : `describe "fonction/arité" do` → `test "cas nominal"` → `test "retourne erreur si..."` → `test "isolé par scope"`
- Fixtures réutilisables dans `test/support/fixtures/` :
`transaction_fixture/2`, `financial_profile_fixture/2`,
`user_scope_fixture/0`
- Chaque test est indépendant : base de données dans un `Sandbox` Ecto, rollback automatique après chaque test

11.3 Tests d'intégration (LiveView)

- `ConnCase` + `register_and_log_in_user` : authentification simulée sans passer par les vrais magic links
- Tests de rendu :
`{:ok, _live, html} = live(conn, ~p"/dashboard") → assert html =~ "Tableau de bord"`
- Tests d'interaction : `live |> element("button", "Valider") |> render_click()` — simule un clic utilisateur
- Tests de formulaire : `live |> form("form", params: %{...}) |> render_submit()` — simule une soumission
- Tests de navigation : `assert has_element?(live_view, "a[href*='income_type=salary']")`

11.4 Tests de sécurité

- Redirect non-authentifié : `live(build_conn(), ~p"/dashboard") → {:error, {:redirect, %{to: "/users/log-in"}}}` — vérifié pour chaque LiveView protégée
- Isolation par scope : créer une transaction pour `other_scope` , vérifier via `refute html =~ "label de l'autre utilisateur"` — chaque LiveView a ce test
- Ces tests fonctionnels valident simultanément le comportement et la sécurité

11.5 Résultats et compte-rendu de tests

- Nombre total de tests et résultat : X tests, 0 failures (insérer la capture du terminal `mix test`)
 - Capture d'écran d'un run CI GitHub Actions réussi
 - Couverture fonctionnelle : lister les fonctionnalités couvertes et celles hors périmètre (fonctions privées, comportement des librairies tierces)
 - Lien vers l'historique des runs CI sur GitHub Actions
-

12. CP10 — Préparer et documenter le déploiement d'une application

12.1 Environnements (développement, test, production)

- Développement (`config/dev.exs`) : base locale, mailer en mode local (pas d'envoi réel), logs détaillés, hot reload
- Test (`config/test.exs`) : base de test séparée, pool `Ecto.Adapters.SQL.Sandbox` (transactions rollbackées automatiquement), mailer en mémoire
- Production (`config/runtime.exs`) : variables d'environnement chargées au démarrage, HTTPS, mailer SMTP configuré, logs structurés

12.2 Procédure de déploiement

⚠ A CRÉER avant le jury : aucun `DEPLOYMENT.md` n'existe dans le projet. Écrire ce fichier à la racine du dépôt avec les étapes complètes, puis s'en servir de base pour rédiger cette section.

- Étapes séquentielles : cloner le dépôt → configurer les variables d'environnement → `mix deps.get --only prod` → `mix ecto.migrate` → `mix assets.deploy` → démarrer le serveur
- Si déployé sur une plateforme (Fly.io, Render, Heroku) : montrer le fichier de configuration (`fly.toml` , `render.yaml`)
- Vérification post-déploiement : vérifier que la page de connexion s'affiche, qu'un magic link peut être demandé

12.3 Scripts de déploiement et migrations

- `mix ecto.migrate` en production : exécuté à chaque déploiement, les migrations sont idempotentes
- `mix release` : génération d'une release OTP autonome (n'embarque pas le runtime Elixir séparément)
- `mix setup` : commande custom qui enchaîne `mix deps.get` , `mix ecto.setup` , `mix assets.setup` , `mix assets.build` — un seul point d'entrée pour l'installation initiale

12.4 Variables d'environnement et configuration

- Liste des variables requises : `SECRET_KEY_BASE` , `DATABASE_URL` , `PHX_HOST` , `PORT`
 - `.env.example` : documentation des variables attendues, committé dans le dépôt (sans valeurs réelles)
 - Règle de configuration : `System.get_env` uniquement dans `config/runtime.exs` , jamais dans le code applicatif — le code utilise `Application.get_env`
 - Avantage : la configuration est centralisée, testable, et ne fuit pas dans les logs
-

13. CP11 — Contribuer à la mise en production dans une démarche DevOps

Dans le cadre de la conteneurisation de ce projet, j'ai été amené à créer un `Dockerfile` et un `docker-compose.yml` qui permettent de lancer l'application et sa base de données dans des conteneurs isolés, sans aucune installation locale d'Elixir ou de PostgreSQL.

Le `Dockerfile` part de l'image officielle `elixir:1.18-alpine` (version figée — utiliser `latest` serait une mauvaise pratique en production car une mise à jour de l'image peut casser le build sans avertissement). Grâce aux commandes fournies par `Mix` évoquées plus tôt, il :

- Installe Hex et Rebar (`mix local.hex && mix local.rebar`), qui sont les gestionnaires de paquets Elixir
- Récupère les dépendances du projet (`mix deps.get`)
- Télécharge les assets frontend (`mix assets.setup`)
- Compile et minifie ces assets (`mix assets.deploy`) pour produire les fichiers statiques prêts pour la production
- Compile l'application complète en environnement de production (`MIX_ENV=prod mix compile`)

Une amélioration à apporter est l'adoption d'un build multi-stage : un premier stage effectue la compilation (avec les outils de build), un second stage ne contient

que la release OTP compilée, ce qui produit une image finale légère sans les dépendances de compilation. Ce point est développé en §13.2.

Lorsque le conteneur est démarré, il joue les fichiers de migration (`mix ecto.migrate`) puis lance l'application (`mix phx.server`).

Ce Dockerfile est lancé à l'aide d'une orchestration docker-compose aux côtés de la base de données pour garantir la reproductibilité.

L'orchestration fonctionne comme suit :

- Le service `db` est créé sur la base de l'image officielle `postgres:16` (version fixée explicitement), avec un volume persistant.
- Un healthcheck vérifie que PostgreSQL est lancé et accepte les connexions.
- Le service `app` est annoté comme dépendant de la condition de santé du service `db`
- Il est construit à partir du Dockerfile local, et charge les variables d'environnement depuis un fichier `.env`.
- La variable `DATABASE_URL` est injectée directement pour pointer vers le service `db` via son nom de service Docker.

Cette configuration permet de lancer l'environnement complet avec une seule commande (`docker compose up`), ce qui garantit la reproductibilité de l'opération indépendamment de la machine hôte.

13.1 Pipeline CI/CD (GitHub Actions)

- Déclencheurs : à chaque push et à chaque PR vers `main`
- Étapes dans l'ordre : format check → Credo strict → compile
 `--warnings-as-errors` → tests ExUnit
- Justification de l'ordre : les vérifications rapides (format) bloquent avant les lentes (tests) — feedback rapide
- Service PostgreSQL 16 dans le runner : les tests s'exécutent contre une vraie base, pas de mock
- Montrer le fichier de workflow `.github/workflows/` et commenter les sections clés

13.2 Conteneurisation (Docker)

⚠ A CRÉER avant le jury : `Dockerfile` et `docker-compose.yml` absents (même lacune que 3.3). Cette section et la 3.3 peuvent être rédigées en parallèle une fois les fichiers créés.

- `Dockerfile` multi-stage : stage builder (compilation Elixir + assets), stage runner (release OTP minimale)
- Avantage du multi-stage : image finale légère, sans outils de build
- `docker-compose.yml` : orchestration de la base PostgreSQL et de l'application pour le développement ou la production
- Si non implémenté : expliquer l'alternative et la compréhension du principe (image OTP autonome, variables d'environnement injectées)

13.3 Outils de qualité de code (Credo, mix format)

- `mix format` : formateur officiel Elixir sans configuration custom — zéro débat de style, appliqué à chaque commit
- `mix credo --strict` : analyse statique couvrant la complexité cyclomatique, les modules sans documentation, les conventions Elixir idiomatiques
- `mix compile --warnings-as-errors` : transforme les avertissements (variables inutilisées, clauses manquantes) en erreurs — aucun warning toléré en CI
- `mix precommit` : alias custom qui enchaîne les trois — habitude de vérification avant chaque commit

13.4 Automatisation des tests

- Les tests s'exécutent automatiquement à chaque push via GitHub Actions — aucune intervention manuelle requise
- En développement : `mix test --failed` pour relancer uniquement les tests en échec sans tout réexécuter
- Isolation via `Ecto.Adapters.SQL.Sandbox` : chaque test reçoit sa propre transaction rollbackée, pas d'interférence entre tests parallèles
- Pas de mock de base de données : les tests s'exécutent contre PostgreSQL réel, ce qui garantit la cohérence avec la production

13.5 Interprétation des rapports CI

- Lire un rapport d'échec GitHub Actions : identifier l'étape échouée, le fichier, la ligne, le message d'erreur
 - Exemple concret à décrire : une CI rouge rencontrée pendant le projet et comment elle a été résolue (ex : test qui attendait une chaîne en anglais après l'activation des traductions françaises)
 - Politique de merge : aucun merge sur `main` si la CI est rouge — garantit que la branche principale est toujours dans un état déployable
-

14. Communiquer en français et en anglais

14.1 Documentation technique en anglais

- `CLAUDE.md` rédigé entièrement en anglais : conventions d'architecture, commandes, règles de développement
- Noms de variables, fonctions et modules en anglais : `list_transactions`, `FinancialProfile`, `current_scope`
- Commentaires de code en anglais (quand nécessaires)
- Messages de commit en français avec scope en anglais : `LQ-50 feat(dashboard): donuts SVG mensuels`

14.2 Lecture de documentation technique en anglais (niveau B1)

- Sources techniques consultées en anglais : `hexdocs.pm` (Phoenix, Ecto, LiveView), guides officiels Phoenix, documentation Tailwind CSS
 - Exemple concret : comprendre le mécanisme des `streams` LiveView depuis la documentation officielle pour l'implémenter dans Lifequest
 - Capacité à résoudre des problèmes en lisant les issues GitHub et les réponses ElixirForum en anglais
-

15. Mettre en œuvre une démarche de résolution de problème

Pour chaque cas : contexte → symptôme observé → hypothèses formulées → expérimentations menées → solution retenue → leçon tirée.

15.1 Refactoring du modèle de données (LQ-5)

Au démarrage du projet, les revenus et les dépenses étaient modélisés dans deux entités distinctes : `IncomeStream` et `Expense`. Cette séparation semblait logique au premier abord, mais rendait les requêtes d'agrégation complexes (solde mensuel, projection) et doublait le code de gestion CRUD.

Le symptôme déclencheur a été la difficulté à calculer un solde mensuel net depuis deux tables de structures différentes. Les requêtes Ecto devenaient des unions fragiles, difficiles à tester et à maintenir.

Après analyse, la solution retenue a été de fusionner les deux entités en une table `Transaction` unifiée avec un champ `direction` (`:income` / `:expense`). Cette migration a simplifié l'ensemble du contexte `Finances`, réduit le code de moitié et rendu les requêtes d'agrégation directes. La leçon : ne pas modéliser deux entités séparées quand un seul discriminant suffit.

15.2 Encodage des apostrophes dans les tests LiveView

Lors de la traduction de l'interface en français (LQ-65), plusieurs tests ont commencé à échouer avec des assertions du type `assert html =~ "S'inscrire"`. Le texte était pourtant visible dans le navigateur.

L'investigation a révélé que `Phoenix.LiveViewTest` retourne du HTML brut dans lequel les apostrophes sont encodées en entité HTML (`'`). L'assertion attendait le caractère littéral, absent du rendu.

La solution : remplacer toutes les assertions contenant des apostrophes par leur équivalent encodé (`"S'inscrire"`). La leçon : dans les tests LiveView, le rendu est du HTML échappé, pas du texte brut — il faut adapter les assertions en conséquence.

15.3 Diagnostic d'une CI rouge inattendue

Après un merge en apparence anodin, la CI GitHub Actions s'est mise à échouer sur l'étape `mix compile --warnings-as-errors`. En local, la compilation passait sans avertissement.

L'analyse du rapport CI a révélé un warning sur une variable non utilisée, introduite par le merge. La variable existait dans un fichier que je n'avais pas recompilé depuis localement (`mix compile` avait gardé le cache). Forcer `mix compile --force` en local a reproduit l'erreur.

La leçon : les warnings masqués par le cache de compilation local sont capturés par la CI, qui compile toujours from scratch. `mix compile --warnings-as-errors` doit faire partie du workflow local via `mix precommit`, pas seulement du pipeline distant.

16. Apprendre en continu : Veille technologique

16.1 Sources suivies

- **Écosystème Elixir/Phoenix** : ElixirForum, doc officielle Hexdocs, changelog des releases Phoenix et LiveView sur GitHub, Fly.io.
- **Frontend & Design** : Chaînes youtube Hyperplexed, BastiUI, Benjamin Code
- **Sécurité web** : Chaîne youtube Fireship, alertes CVE via GitHub Advisory Database pour les dépendances Hex
- **Développement général** : TLDR; & Daily.dev

16.2 Format et méthode

16.3 Exemple concret

17. Conclusion et bilan de compétences

17.1 Bilan technique

Ce projet m'a permis de couvrir l'ensemble du cycle de développement d'une application web : de la conception du modèle de données jusqu'au déploiement conteneurisé, en passant par la construction d'interfaces réactives, la mise en place d'une CI et la rédaction de tests. Les points forts à valoriser sont l'architecture Phoenix LiveView temps réel, la sécurité par conception (magic link, isolation par scope, requêtes paramétrées, CSRF automatique), et la rigueur du processus de qualité (CI, tests, Credo, conventions de commit).

Les lacunes identifiées honnêtement : les maquettes formelles ont été produites après le développement plutôt qu'en amont, la documentation de déploiement est à compléter, et le plan de tests formel (Annexe C) reste à structurer.

17.2 Bilan personnel

Ce projet confirme mon attrait pour les architectures orientées temps réel et la qualité du code. Il m'a confronté à des choix d'architecture structurants (modèle de données unifié, gestion du scope, isolation des couches) que j'aurais difficilement rencontrés sur un projet guidé. Travailler seul sur un sujet personnel m'a appris à formuler des problèmes clairement avant de chercher des solutions, et à défendre des choix techniques justifiés.

Le fait d'avoir choisi une stack peu répandue (Elixir/Phoenix) a aussi été un défi : la documentation et la communauté existent, mais principalement en anglais et en dehors des ressources francophones habituelles.

17.3 Suite et perspectives

À court terme : compléter la mise en production sur une plateforme cloud, finaliser le Dockerfile multi-stage, et ajouter les fonctionnalités envisagées (historique mensuel glissant, conseil automatisé selon le profil financier).

À l'issue de cette formation, je poursuivrai en Mastère Big Data & Intelligence Artificielle à l'ESGI Paris, dans le but d'ajouter une couche analytique à des projets

comme Lifequest : recommandations personnalisées, prévisions basées sur l'historique, modèles d'allocation d'actifs.

18. Annexes

A. Diagramme MCD / MPD

- Insérer le diagramme entité-association avec les cardinalités
- Légende des relations et des types de données principaux

B. Maquettes des interfaces

- Captures des wireframes ou maquettes Figma pour chaque écran principal
- Enchaînement de navigation entre les écrans

C. Plan de tests complet

- Tableau structuré : ID test / fonctionnalité testée / précondition / action / résultat attendu / résultat obtenu / statut (OK/KO)
- Couvrir au minimum : chaque LiveView (rendu, redirect non-authentifié, isolation scope), les fonctions CRUD du contexte Finances

D. Extraits de code commentés

- 3 à 5 extraits représentatifs sélectionnés parmi : un changeset Ecto avec validations, une requête d'agrégation pour les donuts, un `handle_event` LiveView avec pattern matching, une migration avec index
- Chaque extrait accompagné d'une explication de son rôle et des choix techniques

E. Captures d'écran de l'application

- Dashboard avec les donuts SVG et le bloc balance mensuelle
- Page finances avec les transactions groupées par catégorie
- Formulaire de création de transaction
- Page projections financières
- Page simulation de placements

- Page objectifs financiers